

MẠNG NƠ-RON HỒI QUY (RNN)

Từ Hạn Chế của MLP & 1D-CNN đến Nguyên Lý Xử Lý Dữ Liệu Chuỗi có Trạng Thái Ẩn

Nhóm thực hiện:

Văn Thị Mai Linh Nguyễn Văn Minh Lực Giáp Minh Hiếu Nguyễn Nam Hải

Chuyên đề Học Sâu (Deep Learning)

*"Feedforward nhìn thế giới qua từng bức ảnh rời rạc;
RNN thấu hiểu dữ liệu qua dòng chảy liên tục của thời gian."*

Bản chất Dữ liệu dạng Chuỗi (Sequential Data)

Thứ tự xuất hiện quyết định hoàn toàn ngữ nghĩa của dữ liệu

- **Đặc tính trật tự:** Mỗi điểm dữ liệu tại thời điểm t luôn mang mối tương quan mật thiết với các điểm quá khứ $t - 1, t - 2, \dots$
- **Hai dạng chính:** Dữ liệu token rời rạc (Text, DNA) và dữ liệu chuỗi số liên tục (Giá vàng, cổ phiếu, nhịp tim).

Chuỗi Rời rạc (NLP / Văn bản)

”Hà Nội không mưa” $\neq$ ”Mưa không Hà Nội”

→ Câu đầu có nghĩa rõ ràng, câu sau vô nghĩa hoặc biến đổi ngữ nghĩa.

Chuỗi Liên tục (Time-Series)

Giá vàng: $[82.5 \rightarrow 83.1 \rightarrow 84.0 \rightarrow 85.2]$ (Tăng)

Giá vàng: $[85.2 \rightarrow 84.0 \rightarrow 83.1 \rightarrow 82.5]$ (Giảm)

Bài toán Dự đoán Phần tử Kế tiếp (Next Step Prediction)

Mục tiêu: Cho chuỗi quan sát quá khứ $X = (x_1, x_2, \dots, x_t)$, dự đoán giá trị tương lai x_{t+1} .

Bài toán 1: Next Word Prediction (NLP)

Ngữ cảnh: "We" $\rightarrow$ "are" $\rightarrow$ "learning" $\rightarrow$ "AI" $\rightarrow$ [?]
 $\rightarrow$ Target kỳ vọng: "is"

Bài toán 2: Dự báo Chuỗi thời gian (Time-Series)

Cửa sổ trượt (30 ngày lịch sử):
 $[P(t-29), P(t-28), \dots, P(t-1), P(t)] \rightarrow [P(t+1)]$
 $\rightarrow$ Dự báo biến động ngày 31.

Hai Pipeline Tiền xử lý Đặc trưng

Dữ liệu thô không thể đưa trực tiếp vào mạng mà phải đi qua pipeline chuẩn hóa:

Văn bản (Text Pipeline)

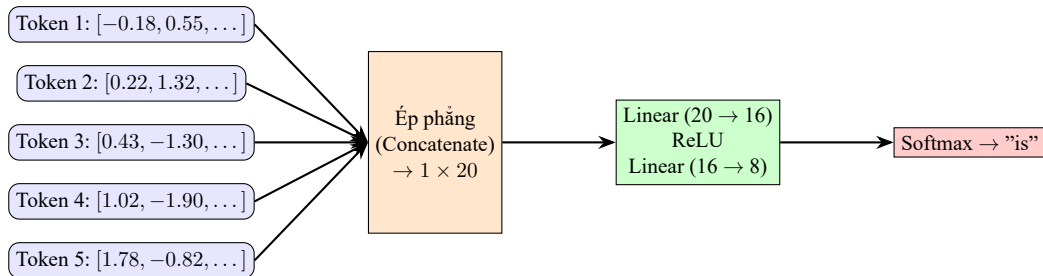
- ❶ Văn bản thô: "giá vàng tăng"
- ❷ Tokenizer: ["giá", "vàng", "tăng"]
- ❸ Vocab Index: [2, 3, 4]
- ❹ Padding: [2, 3, 4, 0, 0]
- ❺ Embedding: Tensor [Batch, Seq_Len, d]

Chuỗi số (Time-Series Pipeline)

- ❶ File CSV thô: Date, Price
- ❷ Làm sạch & Chia tập: Train 70%, Val 15%, Test 15%
- ❸ Scaling: $\log_{1p}(\text{Price}) \rightarrow \text{MinMax}$ (chỉ fit trên Train)
- ❹ Windowing: Cửa sổ trượt 30 ngày
- ❺ Đầu vào: Tensor [N, 30, 1]

Tiếp cận bằng MLP & Cơ chế Ghép phẳng (Flatten)

Mạng truyền thẳng (MLP) bắt buộc phải biến toàn bộ ngữ cảnh thành một vector phẳng duy nhất.



Ví dụ: Tokenizer & Vocabulary cho Next-Word Prediction

1. Code Python: tokenize, chuẩn hóa và tra ID

```
text = "We are learning AI"
tokens = text.split()
tokens = [tok.lower() for tok in tokens]

vocab = {
    "[UNK]": 0, "[PAD]": 1, "ai": 2,
    "a": 3, "are": 4, "cs": 5,
    "is": 6, "learning": 7,
}
ids = [vocab.get(tok, vocab["[UNK]"]) for tok in tokens]
target = vocab["is"]
```

Index (ID)	Word / Token
0	[UNK] (We)
1	[PAD]
2	ai
3	a
4	are
5	cs
6	is (Target)
7	learning

Tokenizer chỉ biến chuỗi thành token; vocabulary biến token thành ID.

2. Kết quả đầu ra

```
tokens = ["we", "are", "learning", "ai"]
ids = [0, 4, 7, 2], target = 6
```

Padding & Embedding: biến ID thành vector số

Code PyTorch: padding và embedding

```
import torch
import torch.nn as nn
import torch.nn.functional as F

ids = torch.tensor([0, 4, 7, 2])
ids = F.pad(ids, (0, 1), value=1)
embedding = nn.Embedding(8, 4)
X = embedding(ids)
```

Kết quả về kích thước

```
ids = [0, 4, 7, 2, 1]
X.shape = torch.Size([5, 4])
```

Ý nghĩa của từng bước

$L = 5$ là cửa sổ đầu vào cố định; [PAD] chỉ được thêm để mọi mẫu có cùng chiều dài.

Nhãn cần dự đoán:

$$y = 6 \iff \text{is}$$

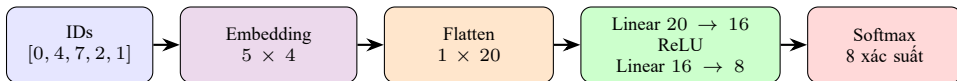
Embedding lookup

Mã trận embedding $E \in \mathbb{R}^{|V| \times d} = \mathbb{R}^{8 \times 4}$.

Mỗi ID được thay bằng một vector học được:

$$\begin{bmatrix} 0 \\ 4 \\ 7 \\ 2 \\ 1 \end{bmatrix} \longrightarrow \begin{bmatrix} e_0 \\ e_4 \\ e_7 \\ e_2 \\ e_1 \end{bmatrix} \in \mathbb{R}^{5 \times 4}$$

MLP dự đoán từ tiếp theo: Forward Pass



Code PyTorch: Flatten và MLP

```
import torch
import torch.nn as nn

X = torch.randn(5, 4)          # output của Embedding
mlp = nn.Sequential(
    nn.Linear(5 * 4, 16), nn.ReLU(),
    nn.Linear(16, 8)
)
x = X.flatten().unsqueeze(0)    # [1, 20]
logits = mlp(x)                 # [1, 8]
```

Kích thước tensor

```
X.shape = [5, 4]
x.shape = [1, 20]
logits.shape = [1, 8]
```

Flatten biến 5 vector embedding thành một vector phẳng; MLP không còn thấy trạng thái theo thời gian.

Từ Softmax đến từ được dự đoán: Loss

Code PyTorch: Softmax và Cross-Entropy

```
import torch
import torch.nn.functional as F
target_probs = torch.tensor([
    [.02, .04, .05, .02, .08, .06, .62, .11]])
logits = target_probs.log()
probs = F.softmax(logits, dim=-1)
y = torch.tensor([6])
loss = F.cross_entropy(logits, y)
print(probs.argmax(dim=-1).item()) # 6 = is
print(loss.item())                 # -log(0.62)
```

Ví dụ đầu ra

Token	Xác suất
ai	0.05
are	0.08
is	0.62
learning	0.11
Các token khác	0.14

$$\hat{y} = \arg \max_i p_i = 6$$

Dự đoán: “is”

Ý nghĩa

$p_i = \text{softmax}(z)_i$, nhân đúng là $y = 6$ (từ is):

$$\mathcal{L} = -\log p_6$$

Muốn loss nhỏ, mô hình phải đẩy xác suất của is lên cao.

Từ giới hạn của MLP đến nhu cầu về RNN

MLP: ĐIỂM NGHẼN

1. Cửa sổ cố định L

Câu dài bị cắt; câu ngắn phải thêm padding.

2. Flatten rời ngữ cảnh

Vị trí 1 và 5 là hai tọa độ độc lập; không có h_{t-1} .

3. Tham số tăng theo L

Flatten: Ld đầu vào $\rightarrow H$ neuron.

$W \in \mathbb{R}^{H \times Ld}$: $H(Ld)$ trọng số.

$b \in \mathbb{R}^H$: H bias $\Rightarrow H(Ld) + H$.

$64(30 \cdot 20) + 64 = 38\,464$ tham số.

bộ nhớ linh hoạt

giữ thứ tự

chia sẻ

RNN: CƠ CHẾ ĐÁP ỨNG

Trạng thái ẩn h_t

Đọc từng bước, giữ ngữ cảnh; nhận chuỗi dài ngắn khác nhau.

Xử lý theo thứ tự thời gian

$h_t = f(x_t, h_{t-1})$: mang theo ngữ cảnh quá khứ.

Chia sẻ trọng số

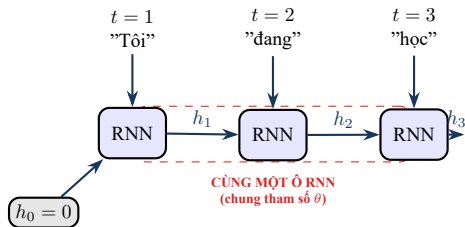
Dùng lại cùng W, U, V ở mọi thời điểm t .

RNN = BỘ NHỚ + THỨ TỰ + CHIA SẺ TRỌNG SỐ

$x_t, h_{t-1} \longrightarrow h_t \longrightarrow y_t$

Ý tưởng Cốt lõi của RNN: Trạng thái Ẩn (h_t)

Trực giác: Con người không bắt đầu tư duy lại từ đầu ở từng chữ, mà liên tục duy trì một dòng ý thức **nén dần thông tin quá khứ**.



h_t = "trí nhớ" nén toàn bộ quá khứ:

- h_1 : chỉ nhớ "Tôi"
- h_2 : nhớ "Tôi" + "đang"
- h_3 : nhớ trọn câu "Tôi đang học"

Phương trình đệ quy của RNN

Mỗi bước chỉ cần **trạng thái liền trước** h_{t-1} và **đầu vào hiện tại** x_t :

$$h_t = f_{\theta}(h_{t-1}, x_t)$$

```
def rnn_cell(x_t, h_prev):  
    a = W @ x_t + U @ h_prev + b_h # tổng nhập  
    return np.tanh(a) # h_t - trí nhớ mới  
h = np.zeros(d_h) # h_0 = 0, chưa biết gì  
for t in range(T):  
    h = rnn_cell(x[t], h) # cập nhật O(1)
```

- **Tóm tắt lịch sử vô hạn:** h_{t-1} đã chứa cả chuỗi quá khứ $x_1, \dots, x_{t-1}$, nên quá khứ dài bao nhiêu cũng được nén vào vector **cố định** d_h chiều.
- **Cập nhật từng bước $O(1)$:** mỗi token đến chỉ cần **một** phép cập nhật; xử lý kiểu streaming, không phải lưu toàn chuỗi trong bộ nhớ.
- **Khác MLP của số trượt:** MLP phải thấy đủ L phần tử trong cửa sổ; RNN duy trì "tóm tắt động" h_t **không phụ thuộc độ dài chuỗi**.

Công thức Toán học – Quá trình Forward

Ba phương trình forward tại bước t , với $x_t \in \mathbb{R}^{d_x}$ (đầu vào), $h_t \in \mathbb{R}^{d_h}$ (trạng thái ẩn), $\hat{y}_t \in \mathbb{R}^{d_y}$ (dự đoán):

$$a_t = \mathbf{W}x_t + \mathbf{U}h_{t-1} + b_h$$

tổng nhập: ghép tin mới với trí nhớ cũ

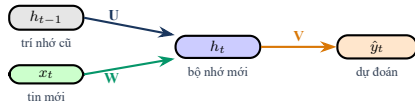
$$h_t = \tanh(a_t)$$

trạng thái ẩn mới, truyền sang bước $t + 1$

$$\hat{y}_t = \mathbf{V}h_t + b_y$$

đọc bộ nhớ h_t để đưa ra dự đoán

Tham số	Kích thước	Kết nối	Vai trò
$\mathbf{W}$	$d_h \times d_x$	Vào $\rightarrow$ Ẩn	Đọc tin mới x_t
$\mathbf{U}$	$d_h \times d_h$	Ẩn $\rightarrow$ Ẩn	Giữ trí nhớ h_{t-1}
$\mathbf{V}$	$d_y \times d_h$	Ẩn $\rightarrow$ Ra	Đọc ra kết quả
b_h, b_y	d_h, d_y	Bias	Dịch ngưỡng



```
a_t = W @ x_t + U @ h_prev + b_h # (1) tổng nhập
h_t = np.tanh(a_t)                # (2) trạng thái ẩn
y_hat = V @ h_t + b_y             # (3) dự đoán
```

- **Vì sao hàm tanh?** Tâm 0, chặn $(-1, 1)$ nên chuẩn của h_t ổn định qua các bước, đạo hàm $1 - \tanh^2(a_t)$ gọn; *sigmoid* lệch dương (trung bình 1/2), đạo hàm ≤ 0.25 làm gradient tắt nhanh; *ReLU* không chặn trên, trạng thái cộng dồn dễ bùng nổ.
- **Tên PyTorch tương ứng:** $\mathbf{W} = \text{weight_ih_l0}$, $\mathbf{U} = \text{weight_hh_l0}$ (Keras: `kernel`, `recurrent_kernel`); mất mát tổng theo thời gian $L = \sum_{t=1}^T \ell(\hat{y}_t, y_t)$ (MSE cho hồi quy).

Ví dụ Tính Toán Forward Từng Bước (số cụ thể)

$$d_x = 2, d_h = 2, d_y = 1; \mathbf{b}_h = \mathbf{0}, b_y = 0.1$$

Bước	Phép tính	Ý nghĩa	Kết quả ($t = 1$)
1	$W \cdot \mathbf{x}_1$	Đọc tin mới Chưa có ký ức	(+0.500, -0.100)
2	$U \cdot \mathbf{h}_0$	(trạng trắng) Ký ức mới	(+0.000, +0.000)
3	$\mathbf{h}_1 = \tanh(W \cdot \mathbf{x}_1 + U \cdot \mathbf{h}_0 + \mathbf{b}_h)$	(vector)	(+0.462, -0.100)
4	$\hat{y}_1 = V \cdot \mathbf{h}_1 + b_y$	Đầu ra (số)	+0.662

Bước $t = 2$: lặp nguyên hệ thống. Thay $\mathbf{x}_2 = (0, 1)$, lấy $\mathbf{h}_1$ vừa tính làm *trí nhớ cũ* (lúc này $U \cdot \mathbf{h}_1 \neq \mathbf{0}$: thành phần hồi quy xuất hiện lần đầu); bộ trọng số (W, U, V) giữ nguyên tuyệt đối, chạy dần đến $t = T$.

$$\text{Shape flow: } \mathbf{x} \in \mathbb{R}^{T \times d_x} \longrightarrow \mathbf{h} \in \mathbb{R}^{T \times d_h} \longrightarrow \hat{\mathbf{y}} \in \mathbb{R}^{T \times d_y}$$

```
W = np.array([[0.5, 0.2],
               [-0.1, 0.4]])
U = np.array([[0.3, 0.0],
               [0.2, 0.5]])
V = np.array([[1.0, -1.0]])
x1 = np.array([1.0, 0.0]); h0 = np.zeros(2)
h1 = np.tanh(W @ x1 + U @ h0 + 0)
# -> [0.462, -0.100] (bước 3)
y1 = (V @ h1 + 0.1)[0]
# -> 0.662 (bước 4)
```

Ghi chú: `PyTorch nn.RNN(2, 2) + nn.Linear(2, 1)` nạp đúng W, U, V , đặt 2 bias của `nn.RNN` về 0 và bias `Linear` = 0.1 thì chạy đúng ví dụ (sai số $< 10^{-6}$).

Nguyên lý Chia sẻ Trọng số (Weight Sharing)

Chuỗi dài 5 bước hay 5.000 bước, RNN đều **dùng chung một bộ ma trận** $\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$ duy nhất:

$$\text{Thời điểm } t = 1: \quad h_1 = \tanh(\mathbf{W} \cdot x_1 + \mathbf{U} \cdot h_0 + b_h) \rightarrow y_1 = \mathbf{V} \cdot h_1$$

$$\text{Thời điểm } t = 2: \quad h_2 = \tanh(\mathbf{W} \cdot x_2 + \mathbf{U} \cdot h_1 + b_h) \rightarrow y_2 = \mathbf{V} \cdot h_2$$

$\vdots$ áp dụng cùng một bộ $\mathbf{W}$, $\mathbf{U}$, $\mathbf{V}$ tới $t = T$

Tổng số tham số P — không chứa T

$$P = \underbrace{d_h \cdot d_x}_{\mathbf{W}} + \underbrace{d_h^2}_{\mathbf{U}} + \underbrace{d_y \cdot d_h}_{\mathbf{V}} + \underbrace{d_h}_{b_h} + \underbrace{d_y}_{b_y}$$

Ví dụ $d_x = 2$, $d_h = 64$, $d_y = 1$:

$$P = 128 + 4096 + 64 + 64 + 1 = 4353 \text{ tham số cho mọi độ dài chuỗi.}$$

So với MLP cửa sổ $L = 30$

- **MLP** ($d_x = 2$, 64 nơ-ron ẩn): trọng số tầng đầu
= $30 \cdot 2 \cdot 64 = 3840$ (+64 bias), chỉ dùng cho **một** độ rộng cửa sổ 30.
- **RNN**: $\mathbf{W} = 2 \cdot 64 = 128$ tham số nhưng **tái sử dụng 30 lần**.

Ba lợi ích cốt lõi

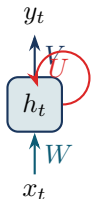
- **Hằng số theo T + tổng quát hóa**: chuỗi dài bao nhiêu, mẫu hình ở đầu hay cuối chuỗi, đều nhận diện giống nhau.
- **Tự điều chuẩn hóa**: ít tham số hơn trên cùng dữ liệu $\rightarrow$ kháng quá khớp tốt hơn.
- **Analogy CNN**: kernel CNN chia sẻ trọng số theo **KHÔNG GIAN**, RNN chia sẻ theo **THỜI GIAN**.

```
rnn = nn.RNN(2, 64)      # W, U, b_h
fc  = nn.Linear(64, 1)   # V, b_y
n = sum(p.numel() for p
        in rnn.parameters())
print(n)  # 4352: W 128 + U 4096
#         + b_h 64+64 (PyTorch tách đôi)
# gộp b_h 1 lần -> công thức P = 4353
```

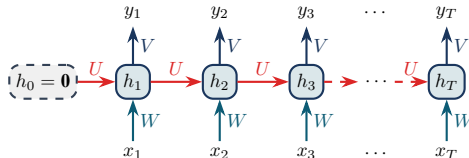
Đồ thị Tính toán dạng Mở cuộn (Unfolded Computation Graph)

Một ô RNN hồi quy khi mở cuộn trở thành một **mạng truyền thẳng cực sâu** có độ sâu đúng bằng T :

DẠNG CUỘN (Folded)



DẠNG MỞ CUỘN (Unfolded)



CÙNG MỘT Ô

— dùng chung W , U , V mọi thời điểm

Mở cuộn = Mạng sâu T tầng

Truyền thẳng **cực sâu**, độ sâu đúng bằng $T \rightarrow$ BPTT chính là backprop trên đúng đồ thị này; T giãn mãi — **tham số cố định**.

U = Đường trí nhớ

Cạnh ngang $h_{t-1} \rightarrow h_t$ (màu đỏ) đưa quá khứ vào hiện tại; **cắt nó** thì mất hết thông tin quá khứ.

Mở cuộn = bỏ đệ quy, viết T bước
 $h = h_0$

```
for t in range(T):  
    # đồ thị sâu T tầng  
    h = np.tanh(W @ x[t] + U @ h + b_h)  
    y[t] = V @ h + b_y
```

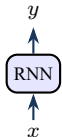
PyTorch: `out, h_n = rnn(X) -> (B, T, 64)`

Các Dạng Kiến trúc Tương tác Chuỗi (Phần 1)

Tùy số lượng bước vào / bước ra, một ô RNN tạo thành các kiến trúc tương tác chuỗi khác nhau:

One-to-One

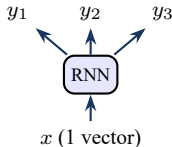
1 bước vào $\rightarrow$ 1 bước ra



Phân loại 1 ảnh — mốc đối chiếu, chưa khai thác ngữ cảnh chuỗi

One-to-Many

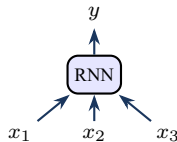
1 bước vào $\rightarrow$ chuỗi ra



Sinh chú thích ảnh (image captioning), sinh nhạc

Many-to-One

chuỗi vào $\rightarrow$ 1 đầu ra



Phân loại cảm xúc review; dự báo giá ngày mai — dạng dùng trong phần thực nghiệm của bài

Many-to-One - dạng asm6

```
rnn = nn.RNN(2, 64, batch_first=True)
```

```
out, h_n = rnn(X)
```

```
y = fc(out[:, -1])
```

```
# đọc chuỗi vào (B, 30, 2)
```

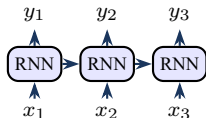
```
# out: (B, T, 64)
```

```
# chỉ đọc bước CUỐI -> 1 dự đoán
```

Các Dạng Kiến trúc Tương tác Chuỗi (Phần 2)

Many-to-Many (Đồng bộ)

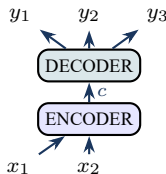
mỗi bước vào $\rightarrow$ 1 nhãn ra



Gán nhãn từ loại POS / NER: mỗi token ra đúng 1 nhãn

Seq2Seq (Encoder-Decoder)

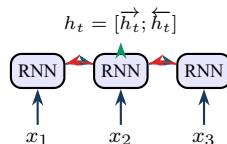
độ dài ra $\neq$ độ dài vào



Encoder nén chuỗi vào thành vector ngữ cảnh c ; Decoder sinh chuỗi ra: dịch máy, tóm tắt

Bidirectional RNN

nhìn cả 2 chiều thời gian

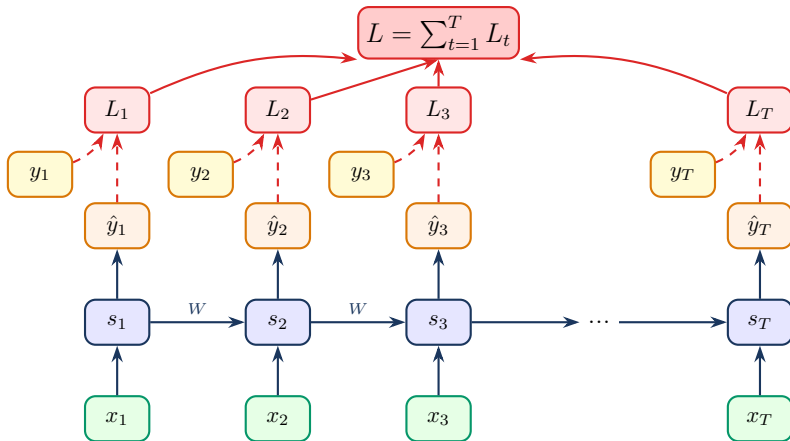


NER, embedding; không dùng dự báo tương lai — nhìn được tương lai = rò rỉ dữ liệu

Liên hệ bài toán asm6 — code PyTorch

```
# Bidirectional: 2 chiều ghép h_t
bi = nn.RNN(2, 64, bidirectional=True)
out, h_n = bi(X) # out: (B, T, 128)
ctx = h_n[-1]    # vector ngữ cảnh c (Seq2Seq)
```

Hàm mất mát (Loss Function) - Minh họa



Backpropagation Through Time (BPTT)

Có 3 tham số ta cần phải tìm là W, U, V . Để thực hiện gradient descent, ta cần tính: $\frac{\partial L}{\partial U}, \frac{\partial L}{\partial V}, \frac{\partial L}{\partial W}$.
Tính đạo hàm với V thì khá đơn giản vì nó chỉ phụ thuộc trực tiếp vào bước hiện tại:

$$\frac{\partial L}{\partial V} = \sum_{t=1}^T \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial V}$$

Tuy nhiên với U, W thì lại khác. Xét tổng quát tại thời điểm T :

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial s_t} \cdot \frac{\partial s_t}{\partial W}$$

Do trạng thái $s_t = \tanh(W \cdot s_{t-1} + U \cdot x_t)$ có s_{t-1} phụ thuộc vào W , ta phải áp dụng Chain Rule truy hồi liên tục về tận thời điểm đầu tiên.

BPTT: Đạo hàm cho W

Sử dụng công thức đạo hàm hàm hợp $(f(x) \cdot g(x))' = f'(x)g(x) + f(x)g'(x)$, ta có:

$$\frac{\partial s_T}{\partial W} = \frac{\partial s'_T}{\partial W} + \frac{\partial s_T}{\partial s_{T-1}} \cdot \frac{\partial s_{T-1}}{\partial W}$$

trong đó $\frac{\partial s'_T}{\partial W}$ là đạo hàm của s_T với W khi coi s_{T-1} là hằng số đối với W .

Tương tự, trong biểu thức s_{T-1} có s_{T-2} phụ thuộc vào W ...nên áp dụng công thức trên và chuỗi Chain Rule, ta được:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \sum_{i=0}^t \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_i} \cdot \frac{\partial s'_i}{\partial W}$$

trong đó $\frac{\partial s_t}{\partial s_i} = \prod_{j=i}^{t-1} \frac{\partial s_{j+1}}{\partial s_j}$ và $\frac{\partial s'_i}{\partial W}$ là đạo hàm của s_i với W khi coi s_{i-1} là hằng số.

BPTT: Đạo hàm cho U

Hoàn toàn tương tự với ma trận trọng số U (từ input vào hidden layer):

$$\frac{\partial L}{\partial U} = \sum_{t=1}^T \sum_{i=0}^t \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_i} \cdot \frac{\partial s'_i}{\partial U}$$

Cốt lõi vấn đề của BPTT

Nhìn vào công thức tính đạo hàm của L với W và U ở trên, ta thấy xuất hiện nhân tử:

$$\frac{\partial s_t}{\partial s_i} = \prod_{j=i}^{t-1} \frac{\partial s_{j+1}}{\partial s_j}$$

Đây chính là nguồn cơn của hiện tượng **Vanishing Gradient** (Triệt tiêu đạo hàm) hoặc **Exploding Gradient** (Nổ đạo hàm) khi chuỗi T lớn.

Chứng minh Toán học: Phân rã Gradient trong RNN

Xét thành phần cốt lõi trong BPTT là đạo hàm trạng thái qua nhiều bước thời gian: $\frac{\partial s_T}{\partial s_k} = \prod_{j=k}^{T-1} \frac{\partial s_{j+1}}{\partial s_j}$.

Do $s_{j+1} = \tanh(W \cdot s_j + U \cdot x_{j+1})$, ma trận Jacobian tại mỗi bước là: $\frac{\partial s_{j+1}}{\partial s_j} = \text{diag}(1 - s_{j+1}^2) \cdot W$.

Đánh giá Chuẩn Ma trận (Matrix Norm)

Áp dụng tính chất $\|A \cdot B\| \leq \|A\| \cdot \|B\|$, ta lấy chuẩn 2 về:

$$\left\| \frac{\partial s_{j+1}}{\partial s_j} \right\| \leq \|\text{diag}(1 - s_{j+1}^2)\| \cdot \|W\| \leq 1 \cdot \|W\| = \|W\|$$

(Vì đạo hàm của tanh lớn nhất bằng 1 nên $\|\text{diag}(1 - s_{j+1}^2)\| \leq 1$).

Cơ chế Vanishing Gradient

Nếu trị riêng lớn nhất của W nhỏ hơn 1 (tức $\|W\| = \gamma < 1$): $\left\| \frac{\partial s_T}{\partial s_k} \right\| \leq \gamma^{T-k} \xrightarrow{T-k \rightarrow \infty} 0$.

⇒ Tín hiệu gradient suy giảm theo hàm mũ về 0. Trọng số mất hoàn toàn khả năng cập nhật theo các lỗi từ quá khứ xa.

Hiện tượng Nổ Gradient (Exploding Gradient)

Cơ chế Toán học (Exploding Gradient)

Ngược lại với triết tiêu, nếu trị riêng lớn nhất của W lớn hơn 1 ($\|W\| = \gamma > 1$) và phần lớn nơ-ron hoạt động ở vùng tuyến tính của tanh (nơi đạo hàm xấp xỉ 1):

$$\left\| \frac{\partial s_T}{\partial s_k} \right\| \approx \gamma^{T-k} \xrightarrow{T-k \rightarrow \infty} \infty$$

Hậu quả Hình học ("Vách đá" Hàm Loss)

Sự nhân dồn liên tục của W qua T bước thời gian tạo ra những "vách đá" (cliffs) cực dốc trong không gian tham số.

- Khi mô hình tiến gần vách đá, đạo hàm bùng nổ theo hàm mũ.
- Gradient khổng lồ gây ra bước cập nhật (Overshooting) đẩy văng mô hình khỏi vùng hội tụ, sinh ra lỗi NaN hoặc Inf.

Giải pháp khắc phục: Gradient Norm Clipping

Để ngăn overshooting, ta hãm phanh bước nhảy. Đặt vector $g = \nabla L = \left[\frac{\partial L}{\partial W}, \frac{\partial L}{\partial U}, \frac{\partial L}{\partial V} \right]$ là vector gradient chứa đạo hàm của tất cả các trọng số trong mô hình.

Cơ chế Cắt tỉa (Clipping)

Nếu độ lớn (norm) của vector g vượt ngưỡng θ , ta co giãn toàn bộ vector:

$$g_{\text{clipped}} = \begin{cases} g & \text{nếu } \|g\| \leq \theta \\ \theta \cdot \frac{g}{\|g\|} & \text{nếu } \|g\| > \theta \end{cases}$$

Tại sao dùng Norm Clipping thay vì Value Clipping?

- **Value Clipping (Cắt từng phần tử):** Ép lẻ từng $g_i \in [-\theta, \theta]$. Việc này **làm bóp méo hướng** của gradient, khiến mô hình cập nhật sai hướng.
- **Norm Clipping (Co giãn vector):** Phép nhân vô hướng với tỷ lệ $\frac{\theta}{\|g\|}$ giúp vector mới tuy ngắn lại nhưng **bảo toàn tuyệt đối hướng đi nguyên thủy** trong không gian đa chiều.

**Lưu ý:* Clipping chỉ chống Nổ đạo hàm, hoàn toàn vô dụng với Triệt tiêu đạo hàm.

Từ RNN đến BiRNN: Mở rộng theo hai hướng

Điểm khác so với RNN một chiều

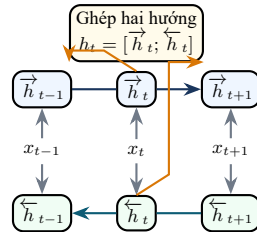
RNN thường chỉ truyền thông tin thuận: $\vec{h}_t = f(x_t, \vec{h}_{t-1})$.

BiRNN thêm nhánh ngược: $\overleftarrow{h}_t = f(x_t, \overleftarrow{h}_{t+1})$, sau đó ghép hai trạng thái:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

Lợi thế chính

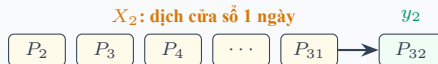
Mỗi vị trí nhận ngữ cảnh từ cả đầu và cuối của **cửa sổ đầu vào**; biểu diễn giàu thông tin hơn RNN một chiều.



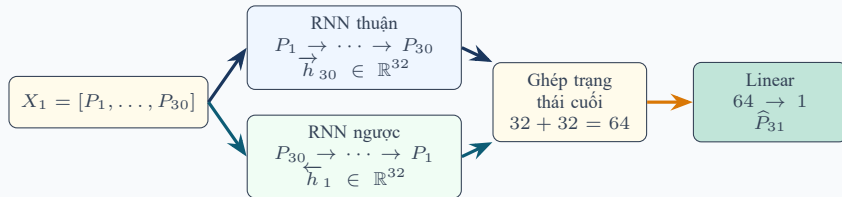
Hai hướng chỉ hoạt động trong cửa sổ dữ liệu đã biết.

BiRNN với giá vàng: Windowing và dự đoán ngày kế tiếp

1. Windowing: tạo các cặp đầu vào–nhãn



2. Một cửa sổ được đọc theo hai hướng



BiRNN hồi quy: từ tensor đến giá dự đoán

Code PyTorch cốt lõi

```
import torch
import torch.nn as nn

birnn = nn.RNN(1, 32, batch_first=True,
               bidirectional=True)
head = nn.Linear(2 * 32, 1)

X_batch = torch.tensor(X[:32], dtype=torch.float32) # [32,
              30, 1]
_, h = birnn(X_batch) # [2,
              32, 32]
h = torch.cat([h[0], h[1]], dim=1) # [32,
              64]
y_hat = head(h) # [32, 1]
```

Đọc các kích thước

[32, 30, 1]: 32 cửa sổ, 30 ngày, 1 giá trị/ngày.

[2, 32, 32]: 2 hướng, 32 mẫu, 32 trạng thái.

Ghép hai hướng: $32 + 32 = 64$.

Đầu ra [32, 1]: mỗi cửa sổ cho một giá dự đoán.

Hồi quy giá vàng

Đầu ra là một số thực $\hat{P}_{t+1}$, nên thường dùng MSE hoặc MAE làm loss.

Điều kiện không lộ dữ liệu tương lai

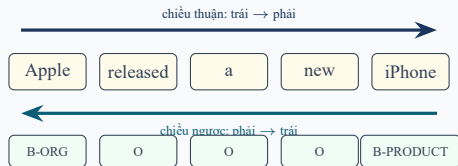
BiRNN chỉ đọc cửa sổ $[P_{t-29}, \dots, P_t]$; giá mục tiêu P_{t+1} chỉ dùng để tính loss, không đưa vào X_batch.

BiRNN trong gán nhãn chuỗi: khai thác ngữ cảnh hai phía

Bài toán phù hợp

BiRNN phù hợp với các nhiệm vụ xử lý **toàn bộ chuỗi đã có sẵn**, trong đó mỗi token cần một nhãn riêng và nhãn đó phụ thuộc cả phần bên trái lẫn bên phải. Ví dụ: nhận dạng thực thể (NER) hoặc gán nhãn từ loại (POS tagging).

Ví dụ: nhận dạng thực thể (NER)



Với token “Apple”, ngữ cảnh bên phải “released a new iPhone” giúp mô hình hiểu đây là một **tổ chức**, không phải một loại quả.

BiRNN tạo nhãn như thế nào?

$$\vec{h}_t = f(x_t, \vec{h}_{t-1})$$

$$\overleftarrow{h}_t = f(x_t, \overleftarrow{h}_{t+1})$$

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

$$\hat{y}_t = \text{softmax}(Wh_t + b)$$

Mỗi vị trí (t) nhận một vector chứa thông tin từ **hai phía**, rồi dự đoán một nhãn riêng cho token đó.

Điểm mạnh và giới hạn

Mạnh: gán nhãn token, NER, POS tagging và phân đoạn chuỗi khi toàn bộ chuỗi đã có.

Giới hạn: không phù hợp với dự báo online/causal, vì hướng ngược cần dữ liệu ở phía tương lai.

TIẾN TRÌNH CÔNG NGHỆ MÔ HÌNH HÓA DỮ LIỆU CHUỖI

MLP (2012)	RNN THUẦN (1986-1990)	BiRNN	TRANSFORMER (2017+)
• Ép phẳng, cửa sổ cố định • Mất tính tuần tự • Phù hợp chuỗi ngắn	• Bộ nhớ ẩn h_t • Bị triệt tiêu gradient • Dễ nổ đạo hàm	• Ngữ cảnh hai chiều trong cửa sổ • Gấp đôi nhánh tính toán • Dự báo offline theo windowing	• Cơ chế Self-Attention • Xử lý song song tuyệt đối • Thống trị lĩnh vực LLM

Thông điệp kết luận

Simple RNN đã đặt nền móng lý thuyết mang tính lịch sử về việc chia sẻ trọng số thời gian và trạng thái ẩn. Hiểu rõ các điểm nghẽn toán học của RNN chính là chìa khóa để làm chủ toàn bộ các mô hình học sâu chuỗi hiện đại.

Phần 2: Tiền xử lý dữ liệu chuỗi cho RNN

Từ dữ liệu thô đến tensor đầu vào

Hai loại chuỗi cần hai pipeline khác nhau.

Văn bản: token rời rạc

Tách từ $\rightarrow$ gán ID $\rightarrow$ padding $\rightarrow$ embedding.

Đầu ra: tensor vector thực theo từng bước thời gian.

Giá vàng: số liên tục

Làm sạch $\rightarrow$ biến đổi và scaling $\rightarrow$ cửa sổ trượt.

Đầu ra: tensor (N, L, F) để dự đoán giá kế tiếp.

Ý chính

Giá vàng đã là dữ liệu số: không cần tokenizer hay vocabulary. Văn bản phải được mã hóa trước khi vào RNN.

Hai loại dữ liệu chuỗi, hai pipeline

Chuỗi văn bản

- ➊ **Tokenizer:** tách câu thành token.
- ➋ **Vocabulary:** ánh xạ token $\mapsto$ ID.
- ➌ **Integer encoding:** tạo chuỗi ID.
- ➍ **Padding:** đưa các câu về cùng độ dài trong batch.
- ➎ **Embedding:** ID $\mapsto$ vector thực.

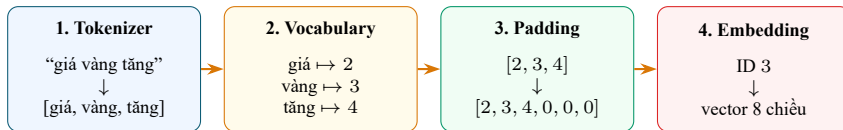
Chuỗi giá vàng

- ➊ **Làm sạch:** date là chỉ mục thời gian, price là số.
- ➋ **Chia theo thời gian:** giữ thứ tự quá khứ $\rightarrow$ tương lai.
- ➌ **Biến đổi:** $\log(1 + p)$ và scaling.
- ➍ **Windowing:** 30 quan sát $\rightarrow$ giá kế tiếp.
- ➎ **Tensor:** $(N, L, F) = (\text{mẫu}, \text{bước}, \text{đặc trưng})$.

Phân biệt quan trọng

Date không phải token; price không cần vocabulary; cửa sổ cố định thường không cần padding.

Tokenizer, vocabulary, padding và embedding



Tầng embedding là bảng tra cứu học được

Với ma trận $E \in \mathbb{R}^{V \times d}$, token ID i lấy vector $E[i] \in \mathbb{R}^d$.

Ví dụ notebook: `token_ids` có shape (4, 6); sau embedding 8 chiều thành (4, 6, 8).

Pipeline tiền xử lý giá vàng

Chuẩn bị và chia tập

- ❶ Đọc CSV: cột date, price.
- ❷ Ép date về ngày; price về số.
- ❸ Bỏ dòng thiếu, bỏ ngày trùng, sắp theo thời gian.
- ❹ Chia train/validation/test theo tỷ lệ 70/15/15.

Biến đổi và tạo mẫu

- ❺ Biến đổi $z_t = \log(1 + p_t)$.
- ❻ Fit scaler *chỉ trên train*, rồi transform các tập.
- ❼ Dùng cửa sổ 30 bước: $X_t = (z_{t-29}, \dots, z_t)$,
 $y_t = z_{t+1}$.

Kích thước trong notebook

13,461 dòng thô → 13,320 dòng sau làm sạch; cửa sổ dài 30.

$$x_{\text{scaled}} = \frac{x - \min_{\text{train}}}{\max_{\text{train}} - \min_{\text{train}}}.$$

Code minh họa: tokenizer và embedding

```
tokens = text.lower().split()
vocab = {'<PAD>': 0, '<UNK>': 1}
# Add frequent training tokens to vocab here.

encoded = [vocab.get(token, vocab['<UNK>'])
            for token in tokens]
padded = encoded[:max_len] + [0] * max(
    0, max_len - len(encoded))

embedding = nn.Embedding(
    len(vocab), embedding_dim=8,
    padding_idx=0)
vectors = embedding(token_ids)
```

Shape qua từng bước

token_ids: (4, 6)

vectors: (4, 6, 8)

Ý nghĩa

Mỗi token ID được thay bằng một vector 8 chiều.
Trục thời gian 6 bước vẫn được giữ nguyên.

Đây là mã rút gọn để minh họa; vocabulary phải được xây từ dữ liệu huấn luyện.

Code minh họa: chuỗi giá vàng

```
df['date'] = pd.to_datetime(df['date'], errors='coerce')
df['price'] = pd.to_numeric(df['price'], errors='coerce')
df = (df.dropna(subset=['date', 'price'])
      .drop_duplicates('date')
      .sort_values('date'))
```

```
raw_prices = df['price'].to_numpy().reshape(-1, 1)
values = np.log1p(raw_prices)
# Split chronologically before fitting scaler.
split_idx = int(0.70 * len(values))
values_train = values[:split_idx]
scaler.fit(values_train)
scaled_train = scaler.transform(values_train)
# Build windows of 30 past observations.
```

Chuỗi số liên tục đi thẳng vào mô hình; không dùng tokenizer hay embedding.

Tensor đưa vào RNN

X_train: (9294, 30, 1)

9294 mẫu; 30 bước thời gian; 1 đặc trưng.

Nhãn dự đoán

y_train: (9294, 1)

30 quan sát quá khứ → giá bước kế tiếp.

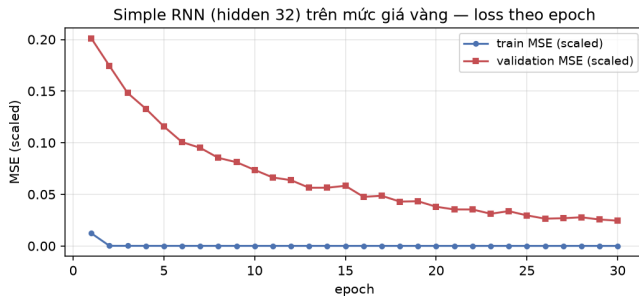
Simple RNN: mô hình và quá trình huấn luyện

```
class
def
    super

def
    return
```

Luồng tensor

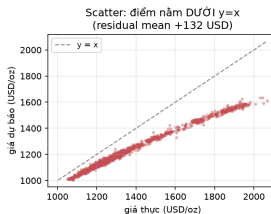
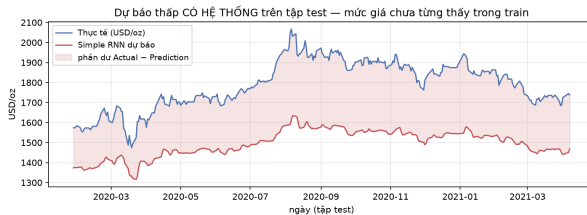
$(B, 30, 1) \rightarrow (B, 32) \rightarrow (B, 1)$



Đọc đồ thị loss

Train loss gần 0; validation loss giảm mạnh rồi ổn định ở mức thấp. Đây là MSE trên dữ liệu đã scale, nên vẫn phải đánh giá sai số trên giá gốc và so với baseline.

Dự báo và phần dư trên tập test



Đọc kết quả: RNN chưa vượt baseline

Metrics trên tập test (theo notebook)

Mô hình	MAE	RMSE	MAPE	Direction
Persistence	9.2163	13.4675	0.6714%	77.3273%
Simple RNN	236.1134	252.1612	16.9713%	49.1992%

Dịch chuyển phân phối

Train max: **850.00**; test min: **1,049.40**.

100% giá test vượt train max.

Dấu hiệu trên biểu đồ

Persistence bám sát giá thực. RNN dự báo thấp có hệ thống: phần dư *Actual* – *Prediction* dương; điểm phân tán nằm dưới đường $y = x$.

Hướng cải thiện

Thử dự báo log-return thay cho mức giá, huấn luyện bằng cửa sổ gần thời điểm test hoặc thử GRU/LSTM; luôn so sánh lại với Persistence trên tập test theo đúng thứ tự thời gian.